So sánh Faster R-CNN ResNet50-FPN và YOLOv8s trong phát hiện Varroa trên ảnh ong mật

Le Khoi Nguyen
Giảng viên hướng dẫn: Phan Thi Thu Hong
Subject: Lab3-ObjectDetection-Varroa/DPL302m
Ngày hoàn thiện báo cáo: 25/10/2026

Tóm tắt. Bài lab xây dựng, huấn luyện và đánh giá hai hệ thống phát hiện đối tượng một lớp: Faster R-CNN ResNet50-FPN và YOLOv8s, nhằm xác định hộp bao Varroa trên ảnh crop ong mật. Sau kiểm toán dữ liệu, bộ thực nghiệm gồm 13.507 ảnh RGB, 4.626 hộp hợp lệ, giữ nguyên train/validation/test của nguồn. Hai hệ thống dùng cùng adapter dữ liệu, letterbox 640×640, augmentation, seed 42 và bộ đánh giá COCO. Trên 3.408 ảnh test, Faster R-CNN đạt AP@[.50:.95] 28,78%, AP50 83,43% và F1 80,19%; YOLOv8s lần lượt đạt 21,93%, 71,16% và 74,06%. Inference FP32, batch 1 trên một Tesla T4 đạt 14,84 FPS và 50,51 FPS. Faster R-CNN phát hiện đúng thêm 103 đối tượng với số false positive gần tương đương, trong khi YOLOv8s nhanh hơn 3,40 lần. Cả hai còn hạn chế về localization ở IoU cao. Báo cáo trình bày đầy đủ chuẩn bị dữ liệu, lựa chọn model, triển khai DDP, xử lý lỗi số học, chọn checkpoint/ngưỡng validation, inference ảnh mới và các giới hạn của so sánh một seed với hai recipe huấn luyện khác nhau.

Từ khóa: Varroa, object detection, Faster R-CNN, ResNet50-FPN, YOLOv8s, COCO evaluation, reproducibility.

1 Giới thiệu và phát biểu bài toán

1.1 Yêu cầu Lab 3

Yêu cầu của bài lab là so sánh và nhận xét họ R-CNN với họ YOLO trên bộ dữ liệu Varroa cho object detection, cho phép lựa chọn một đến hai phiên bản mỗi họ. Thực nghiệm này chọn một đại diện mỗi họ để đủ thời gian kiểm tra toàn bộ quy trình: Faster R-CNN ResNet50-FPN và YOLOv8s. Mục tiêu không chỉ là tạo một bảng điểm mà còn chứng minh dữ liệu, preprocessing, cách chọn model và protocol đánh giá có thể truy vết.

Với ảnh I, đầu ra cần tìm là một tập detection

$$D(I) = \{(b_i, s_i, c_i)\}_{i=1}^{n}, \tag{1}$$

trong đó b_i là hộp bao, $s_i \in [0, 1]$ là confidence và c_i là lớp Varroa. Một ảnh có thể có nhiều Varroa hoặc không có đối tượng nào. Vì vậy, đây là bài toán định vị nhiều instance một lớp, không phải chỉ phân loại ảnh ong khỏe/nhiễm. Không sinh hộp cho toàn bộ con ong và không bổ sung lớp foreground "healthy".

1.2 Bối cảnh dữ liệu và các câu hỏi nghiên cứu

VarroaDataset được thu trong điều kiện ghi hình có kiểm soát, chứa ảnh ong mật 160×280 pixel và split được xác định trước [1, 2]. Trong ảnh, mục tiêu có thể nhỏ, ít tương phản với thân ong và bị ảnh hưởng bởi cánh, texture, tư thế hoặc bóng sáng. Các đặc điểm nhìn thấy trên ảnh chỉ là quan sát định tính; thực nghiệm không gán nguyên nhân sinh học cho từng lỗi của detector.

Bài lab tập trung vào bốn câu hỏi: (i) hệ thống nào đạt AP và recall cao hơn trên cùng test; (ii) khác biệt localization thể hiện thế nào khi tăng IoU; (iii) mức đánh đổi accuracy–speed và chi phí tài nguyên; (iv) pipeline inference ảnh mới có duy trì đúng preprocessing và threshold của evaluation không.

Bảng 1: Dữ liệu sau kiểm toán; "âm" nghĩa là không có hộp Varroa được gán nhãn.

Split	Ảnh	Dương	Âm	Hộp
Train	8.223	2.553	5.670	2.903
Validation	1.876	451	1.425	655
Test	3.408	942	2.466	1.068
Tổng	13.507	3.946	9.561	4.626

1.3 Phạm vi đóng góp

Đóng góp thực hành gồm bộ dữ liệu COCO/YOLO dùng chung đã kiểm toán; hai notebook Kaggle độc lập; evaluator thống nhất và threshold chọn trên validation; module inference chạy ảnh/thư mục; cùng report, hình và số liệu sinh từ artifact thật. Không đề xuất kiến trúc mới, không báo cáo state of the art, và không kết luận mọi two-stage detector đều tốt hơn mọi one-stage detector từ hai run này.

2 Dữ liệu và kiểm toán annotation

2.1 Nguồn, thống kê và split

Nguồn được dùng là VarroaDataset v1.2.0, DOI 10.5281/zenodo.4085044. Thống kê công bố ban đầu có 13.509 mẫu; audit local cho thấy hai dòng train trùng đường dẫn ảnh. Sau khử lặp, tập thực nghiệm có 13.507 ảnh duy nhất. Bảng 1 là thống kê sau xử lý, không sao chép nguyên số dòng CSV thành số ảnh.

Tất cả ảnh giữ nguyên độ phân giải và byte PNG nguồn khi xuất dữ liệu. Ảnh âm chiếm khoảng 70,79% toàn bộ tập, do đó accuracy phân loại ảnh hoặc số ảnh không báo object không đủ đánh giá detector. Việc giữ negative samples cần thiết để đánh giá false alarm, thay vì chỉ đo trên các ảnh đã biết có Varroa.

Giữ nguyên split của nguồn, không trộn frame rồi chia ngẫu nhiên. Các split gồm 6, 5 và 2 nhóm tên video tương ứng train, validation, test. Không có nhóm tên trùng giữa split. Tuy nhiên, khác tên video không chứng minh hoàn toàn độc lập về phiên quay hoặc điều kiện thu thập; đây là giới hạn cần giữ khi diễn giải khả năng tổng quát hóa.

2.2 Ánh xạ nhãn và giải quyết bất nhất

Trường đầu tiên trong TXT nguồn là flag ở mức ảnh, không phải class YOLO hay số lượng box. Flag 0 tương ứng nhãn rỗng; các hộp của flag 1 và 3 được gộp vào một lớp Varroa. Ý nghĩa chi tiết phân biệt flag 1 và 3 chưa được xác nhận từ mô tả gốc; cách gộp này là quyết định của bài lab, dựa trên annotation và kiểm tra hình, không phải một khẳng định về ý nghĩa flag do tác giả dataset cung cấp. Flag được giữ trong manifest để truy vết.

YOLO dùng class 0; COCO và Torchvision dùng category/label 1 cho Varroa, còn 0 của Torchvision dành cho background. Nhãn negative là tensor box hình $(0, 4)$ cùng tensor label rỗng; không thêm box giả để tránh lỗi loader.

Hai trường hợp train cần xử lý là một ảnh dương có hai biến thể hộp cho cùng đối tượng, và một ảnh âm có hai dòng giống nhau. Chọn bản nhãn chuẩn dưới thư mục `labels/`, không hợp hai biến thể thành hai Varroa. Ngoài ra, một box có $x_1 = x_2 = -0,5$ bị loại vì chiều rộng bằng 0; ảnh vẫn giữ box hợp lệ còn lại. Số box train giảm từ 2.905 bộ tọa độ CSV xuống 2.903. Không có box hợp lệ nào cần clip biên. Validation và test giữ nguyên số ảnh và box.

2.3 Định dạng xuất và kiểm tra độc lập

Hai mô hình đọc cùng ảnh trong `images/<split>/<video>/`. Nhãn YOLO là tâm và kích thước chuẩn hóa; COCO lưu hộp pixel (x, y, w, h), area wh, category 1, `iscrowd=0`. Image/annotation ID là duy nhất, đường dẫn tương đối với dataset root. Manifest lưu đường dẫn nguồn, split, flag, kích thước, số box, SHA-256 file và pixel RGB.

Các kiểm tra bao gồm CRC archive; giải mã toàn bộ ảnh; tham chiếu COCO; tọa độ hữu hạn, box dương và nằm trong ảnh; không thiếu/thừa nhãn; và chuyển YOLO ngược về COCO với sai số nhỏ hơn 10^{-6} pixel. Oracle evaluation dùng chính ground truth làm prediction cho AP50, AP75 và AP@[.50:.95] bằng 1. Đây là kiểm tra định dạng/evaluator, không phải kết quả model học được.

Không phát hiện ảnh trùng pixel RGB hoặc cặp cross-split có khoảng cách Hamming dHash 64-bit không quá 3. Phép sàng lọc này không loại trừ mọi tương quan thời gian. Các tệp audit JSON và báo cáo chuẩn bị dữ liệu được giữ cùng repo.

2.4 Phân bố kích thước đối tượng

Theo diện tích ảnh gốc, train có 1.333 small và 1.570 medium box; validation có 358 và 297; test có 768 và 300. Không có large box. Các nhóm tuân theo ngưỡng COCO 32^2 và 96^2 pixel diện tích. Vì vậy, AP large không có ground truth để đo và được xuất `null`, không diễn giải thành hiệu năng 0%. Median chiều rộng/chiều cao box test khoảng 29,74/27,58 pixel; resize lên 640 chỉ nội suy, không tạo thêm chi tiết thị giác từ nguồn 160×280.

3 Lựa chọn mô hình và cơ sở phương pháp

3.1 Faster R-CNN ResNet50-FPN

Faster R-CNN là detector hai giai đoạn: backbone trích đặc trưng; RPN tạo proposal; RoI head phân loại và hồi quy hộp. Chia sẻ đặc trưng giữa RPN và detection head thay thế proposal thủ công bằng một thành phần học được [3]. ResNet50 dùng residual connections để hỗ trợ tối ưu backbone sâu [4]. FPN kết hợp các mức đặc trưng bằng đường top-down và lateral connections, tạo biểu diễn đa tỉ lệ [5]. Đây là cơ sở chọn baseline cho đối tượng có diện tích small/medium, không phải bảo đảm trước rằng model sẽ thắng trên Varroa.

Implementation dùng `fasterrcnn_resnet50_fpn` của Torchvision, khởi tạo COCO V1, thay `FastRCNNPredictor` bằng head hai lớp: background và Varroa. Backbone dùng FrozenBatchNorm theo graph pretrained. Tổng số parameter của model sau thay head là 41.299.161; số này là toàn bộ parameter, không đồng nghĩa toàn bộ đều được cập nhật khi fine-tune.

Loss huấn luyện là tổng các thành phần do Torchvision trả về:

$$\mathcal{L}_F = \mathcal{L}_{cls} + \mathcal{L}_{box} + \mathcal{L}_{obj} + \mathcal{L}_{rpn_box}. \tag{2}$$

Các thành phần lần lượt giám sát phân loại RoI, hồi quy box RoI, objectness và hồi quy proposal. Bài lab dùng loss gốc của framework, không thay thế mục tiêu tối ưu hay thực hiện ablation riêng từng thành phần.

3.2 YOLOv8s

YOLOv8s đại diện hướng one-stage, dự đoán từ đặc trưng đa tỉ lệ mà không tách RPN–RoI như

Faster R-CNN. Cấu hình Ultralytics v8.3.228 dùng các khối C2f, SPPF, phép ghép đặc trưng và detection head P3/P4/P5 với stride 8/16/32; head theo hướng anchor-free [6, 7]. Chọn kích thước `s` để cân bằng tài nguyên và năng lực biểu diễn trong ngân sách lab; không khảo sát đủ `n/m/l/x` để chứng minh `s` tối ưu.

Model khởi tạo từ `yolov8s.pt` COCO-pretrained, fine-tune một lớp. Tổng số parameter sau cấu hình một lớp là 11.135.987. Recipe sử dụng model EMA do Ultralytics quản lý. Loss gồm box, classification và distribution focal loss, với hệ số cấu hình lần lượt 7,5; 0,5; 1,5 [8]. Không thêm một objectness loss riêng bằng cách suy diễn từ YOLO đời cũ. Giá trị loss của YOLO và Faster R-CNN khác định nghĩa và scale, nên không so sánh trực tiếp để xếp hạng model.

3.3 Ý nghĩa của phép so sánh

Các yếu tố được đồng nhất là dữ liệu, seed, input size, augmentation, evaluator, quy tắc chọn checkpoint, chọn threshold và phạm vi đo latency. Các yếu tố khác nhau gồm kiến trúc, normalization nội bộ, learning rate, loss, EMA và precision training. Kết quả vì vậy mô tả hai hệ thống fine-tune cụ thể. Không quy toàn bộ chênh lệch AP cho một mình cơ chế one-stage/two-stage.

4 Triển khai hệ thống

4.1 Cấu trúc mã nguồn

`training/common.py` chứa dataset adapter, letterbox, đổi box, prediction, COCO metrics, chọn ngưỡng, benchmark và final evaluation. `training/fasterrcnn_train.py` triển khai loop Torchvision; `training/yolov8_train.py` mở rộng `DetectionTrainer`. Script `tools/build_training_notebooks.py` nhúng toàn bộ implementation vào hai notebook Kaggle độc lập. Người chạy chỉ cần attach dataset, chọn T4 $\times 2$, bật Internet và Run All; các module runtime được ghi vào `/kaggle/working/`.

Cách nhúng code giúp mỗi notebook tự chạy, nhưng yêu cầu sinh lại notebook khi thay source. Test kiểm tra code trainer được nhúng đúng phiên bản hiện tại. Bản cấu hình, source runtime, package freeze và SHA-256 annotation được lưu trong artifact của run.

4.2 Tiền xử lý và hình học box

Với ảnh có chiều rộng W, chiều cao H và canvas cạnh $S = 640$:

$$r = \min(S/W, S/H), \tag{3}$$

$$W' = \text{round}(Wr), \quad H' = \text{round}(Hr), \tag{4}$$

$$l = \lfloor (S - W')/2 \rfloor, \quad t = \lfloor (S - H')/2 \rfloor. \tag{5}$$

Ảnh RGB resize bằng bilinear rồi đặt vào canvas RGB giá trị 114, chuyển CHW float và chia 255. Tọa độ dùng scale thực $s_x = W'/W$, $s_y = H'/H$:

$$x' = s_x x + l, \quad y' = s_y y + t. \tag{6}$$

Khi inference, lấy $(x' - l)/s_x$ và $(y' - t)/s_y$, clip về $[0, W] \times [0, H]$; loại detection suy biến sau clip. Dùng scale sau làm tròn tránh sai lệch tích lũy.

Với ảnh 160×280, vùng resize là 366×640, padding trái/phải 137 pixel. Faster R-CNN đặt min/max size nội bộ cùng bằng 640 để không resize canvas lần nữa; normalization mean/std của pretrained Torchvision vẫn được giữ. YOLO không chia 255 lần thứ hai trong `preprocess_batch`.

4.3 Augmentation và adapter YOLO

Train dùng lật ngang xác suất 0,5; brightness, contrast và color có hệ số uniform $[0,9; 1,1]$. Box lật theo $x'_1 = W - x_2$, $x'_2 = W - x_1$. Validation/test không augmentation. Không mosaic, mixup, crop, scale ngẫu nhiên, rotation hay multi-scale.

`YOLOCocoDataset` kế thừa adapter chung, chuyển xyxy sang tâm–kích thước chuẩn hóa để cấp cho loss YOLO, tạo `batch_idx` theo ảnh. Augmentation mặc định liên quan của Ultralytics bị vô hiệu hóa để không chồng pipeline thứ hai. Một số giá trị trong `args.yaml` là default framework; hành vi thật cần đọc subclass và evaluator chung, không suy ra chỉ từ file args.

4.4 DDP, workers và tính tái lập

Hai notebook khởi chạy `torch.distributed.run` với hai process. Mỗi process dùng một GPU, NCCL đồng bộ gradient; global batch 8 tương ứng 4 ảnh/GPU. `DistributedSampler` shuffle với seed 42 và cập nhật epoch. DataLoader dùng hai worker/GPU, cơ chế `spawn`, pin memory, seed cho Python/NumPy/Torch. Chỉ rank 0 chạy validation đầy đủ; kết quả fitness và tín hiệu dừng được broadcast.

Seed và cấu hình cuDNN được cố định, nhưng không ép toàn bộ thuật toán deterministic vì cần giữ kernel detection native. Không bảo đảm tái lập giống từng bit trên GPU hay sau resume. Chỉ một seed đã chạy nên không có độ lệch chuẩn giữa các seed.

5 Thiết lập và quá trình huấn luyện

5.1 Môi trường và hyperparameter

Các run chính dùng hai Tesla T4, Python 3.13.15, Torch 2.11.0+cu128, Torchvision 0.26.0+cu128, CUDA 12.8, pycocotools 2.0.11; YOLO dùng Ultralytics 8.3.228. Bảng 2 phân biệt cấu hình chung và khác biệt recipe.

Với Faster R-CNN, multiplier warmup tăng từ 0,01 lên 1 theo step, sau đó cosine tiến về 0,01 lần learning rate cơ sở trong ngân sách 100 epoch. Scheduler chỉ tiến khi update optimizer được thực hiện. YOLO dùng warmup native cho learning rate, bias learning rate và momentum; dù cùng ba epoch warmup, không tuyên bố hai schedule giống hệt. Gradient Faster R-CNN được clip norm ở 10. YOLO giữ các xử lý tối ưu gốc của phiên bản đã pin.

5.2 GPU preflight

Trước run đầy đủ, notebook tạo tập train nhỏ 8 ảnh, cân bằng dương/âm, và 4 ảnh validation; chạy một epoch với đúng GPU, precision, input size và worker. Output smoke tách riêng, không đọc test và không tiếp tục weights smoke sang run chính. Preflight chỉ kiểm tra tính chạy được trên một batch, không đủ chứng minh mọi batch và mọi step của 100 epoch đều ổn định số học.

5.3 Sự cố AMP và biện pháp xử lý

Run Faster R-CNN đầu tiên bật AMP đã qua preflight nhưng dừng ở epoch 1, step 841/1.028: `Non-finite loss`. Code dùng all-reduce MIN của cờ finite nên cả hai rank cùng dừng, kể cả khi chỉ một rank có loss hỏng. Các lỗi `ChildFailedError`, SIGTERM và cảnh báo chưa đóng process group là hệ quả sau lỗi chính.

Log cũ không ghi loss component hay image ID batch lỗi nên chưa xác định phép toán gây NaN/Inf. Audit 2.903 train box không phát hiện tọa độ không hữu hạn, box bằng 0, âm hoặc vượt biên; điều đó loại một số lỗi annotation phổ biến nhưng không chứng minh mọi nguyên nhân còn lại đều do FP16.

Biện pháp triển khai là chuyển Faster R-CNN mặc định sang FP32; kiểm tra từng loss và gradient norm trước optimizer update; nếu lỗi, lưu `nonfinite_rank*.json` gồm rank, epoch/step, loss, learning rate và image ID/box; đóng process group bằng `finally`. Đồng thời chỉ rõ device NCCL khi khởi tạo. Không bỏ qua batch lỗi, không thay nhãn để làm loss hữu hạn. Run FP32 hoàn thành cho thấy recipe sửa đã chạy ổn định trong thực nghiệm này, nhưng không phải bằng chứng nhân quả tuyệt đối về lỗi FP16 do chưa có ablation tái hiện batch hỏng.

5.4 Checkpoint, resume và điểm dừng

Faster R-CNN lưu state model, optimizer, scheduler, scaler, epoch, history và cấu hình; ghi tạm rồi đổi tên để giảm nguy cơ checkpoint ghi dở. YOLO giữ checkpoint đầy đủ và EMA theo trainer. `last.pt` dùng tiếp tục run bị gián đoạn; `best.pt` dùng evaluation/inference. Resume yêu cầu giữ các tham số ngân sách và best checkpoint của run trước; không cam kết khôi phục chính xác trạng thái random của worker. Hai run được báo cáo không resume.

Faster R-CNN hoàn thành 22 epoch, best epoch 7, tổng 21.998,54 giây; YOLOv8s hoàn thành 44 epoch, best epoch 29, tổng 8.476,35 giây. Khoảng cách từ best đến cuối đều là 15 epoch, khớp early stopping. Đây không phải run thất bại vì chưa đạt 100 epoch: 100 là trần ngân sách. Thời gian lấy từ `training_summary.json`; không gồm toàn bộ thời gian tải dataset/dependency hoặc final test/benchmark của notebook, và không cộng run Faster R-CNN AMP thất bại trước đó.

6 Giao thức đánh giá

6.1 COCO AP và hậu xử lý thống nhất

Evaluation dùng pycocotools COCOeval [9, 10], cùng annotation gốc cho cả hai model. Ngưỡng score floor cho AP là 0,001; NMS IoU 0,5; tối đa 100 detection/ảnh. YOLO raw head output được giải mã/NMS qua predictor chung, không dùng evaluator native làm bảng điểm cuối. Tham số NMS native khác trong `args.yaml` không ghi đè protocol cuối này. Box đều đưa về pixel ảnh gốc trước khi đánh giá.

IoU giữa hai hộp được xác định bởi

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}. \tag{7}$$

AP50 và AP75 đo ở IoU 0,50 và 0,75; chỉ số chính lấy trung bình AP qua 10 mức IoU:

$$\mathrm{AP}_{.50:.95} = \frac{1}{10} \sum_{k=0}^{9} \mathrm{AP}_{0.50+0.05k}. \tag{8}$$

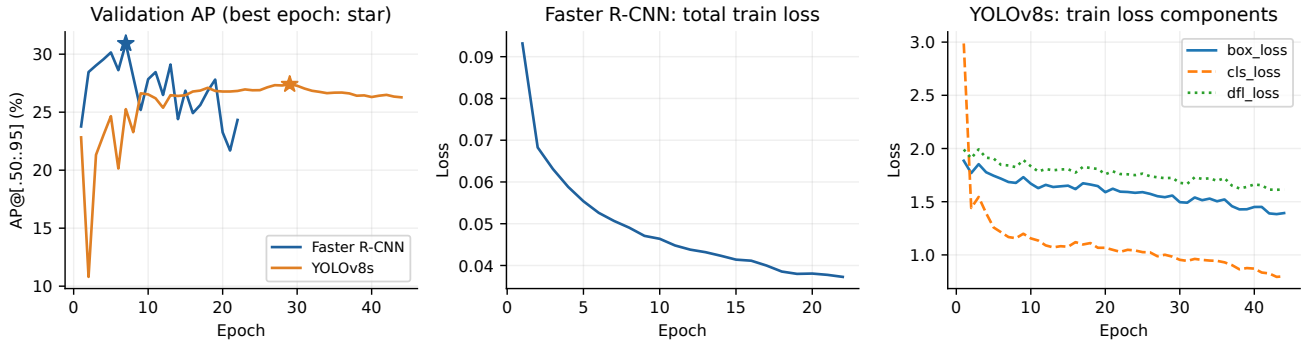
Vì chỉ có một lớp, mAP qua lớp trùng AP của Varroa. AP được tổng hợp theo chuẩn COCO, không phải accuracy ảnh hoặc AP tại một confidence duy nhất.

6.2 Threshold, precision, recall và F1

Threshold vận hành τ được chọn trên toàn bộ validation bằng F1 lớn nhất ở IoU 0,5, hòa điểm ưu tiên confidence lớn hơn. Ghép prediction theo score giảm dần, cùng lớp và cùng ảnh; một ground truth chỉ được nhận tối đa một TP. Detection trùng hộp

Bảng 2: Recipe của hai run chính. Early stopping theo COCO AP validation, không theo test.

Thành phần	Faster R-CNN ResNet50-FPN	YOLOv8s
Khởi tạo	COCO V1 pretrained	COCO pretrained yolov8s.pt
Ảnh / seed / global batch	640×640 / 42 / 8	640×640 / 42 / 8
Optimizer / momentum / decay	SGD / 0,9 / 0,0005	SGD / 0,9 / 0,0005
Learning rate cơ sở	0,005	0,01
Warmup / schedule	3 epoch, tuyến tính; cosine	3 epoch native; cosine
Ngân sách / patience	Tối đa 100 epoch / 15	Tối đa 100 epoch / 15
Precision training	FP32 sau sửa lỗi AMP	AMP FP16
Weights validation/test	Trực tiếp từ model	EMA
Worker / eval batch	2 worker mỗi GPU / 4	2 worker mỗi GPU / 4
Augmentation	Lật ngang, jitter nhẹ qua adapter	Giống Faster R-CNN qua adapter



Hình 1: Validation AP và train loss của các run thực tế. Dấu sao là best epoch. Các loss được vẽ tách panel vì khác định nghĩa/scale; đường loss không dùng để so sánh trực tiếp chất lượng hai kiến trúc.

Bảng 3: Kết quả tiến trình training của các run thành công.

Chỉ số	Faster R-CNN	YOLOv8s
Epoch hoàn thành	22	44
Best epoch	7	29
Best val AP (%)	30,92	27,41
Thời gian (giờ)	6,11	2,35
Số GPU	2	2

đã ghép được tính FP. Các prediction bằng score được xét cùng mức threshold, tránh chọn ngưỡng không thể thực hiện tại thời điểm inference. Sau khi chọn, khóa τ trước khi đọc test predictions.

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \tag{9}$$

$$F_1 = \frac{2PR}{P + R}. \tag{10}$$

Các giá trị default tìm được là $\tau_F = 0,8224449754$ và $\tau_Y = 0,2831989825$. Không so sánh calibration chỉ từ hai số confidence khác nhau; chúng không cùng thang xác suất đã được hiệu chuẩn. AP vẫn tính ở score floor thấp, không cắt bằng τ đã chọn cho P/R/F1.

Mean IoU TP là trung bình IoU của các hộp được ghép đúng ở threshold trên, cần đọc cùng recall vì không phản ánh hộp bị bỏ sót. False alarm trên ảnh âm là tỷ lệ ảnh không có ground truth nhưng có ít nhất một detection; FP/image chia tổng số FP cho tất cả ảnh. Hai chỉ số này không tương đương vì FP còn có thể xuất hiện trên ảnh dương.

6.3 Latency và FPS

Benchmark chạy riêng trên một T4, FP32, batch 1; 20 lượt warmup, 100 ảnh lấy theo chỉ số đều trên test. Ảnh đã được đọc/giải mã trước phép đo. CUDA được synchronize trước/sau mỗi mẫu. Phạm vi gồm letterbox, chuyển CPU–GPU, model, NMS, đổi box về ảnh gốc và chuyển kết quả về CPU; không gồm file I/O/giải mã ảnh.

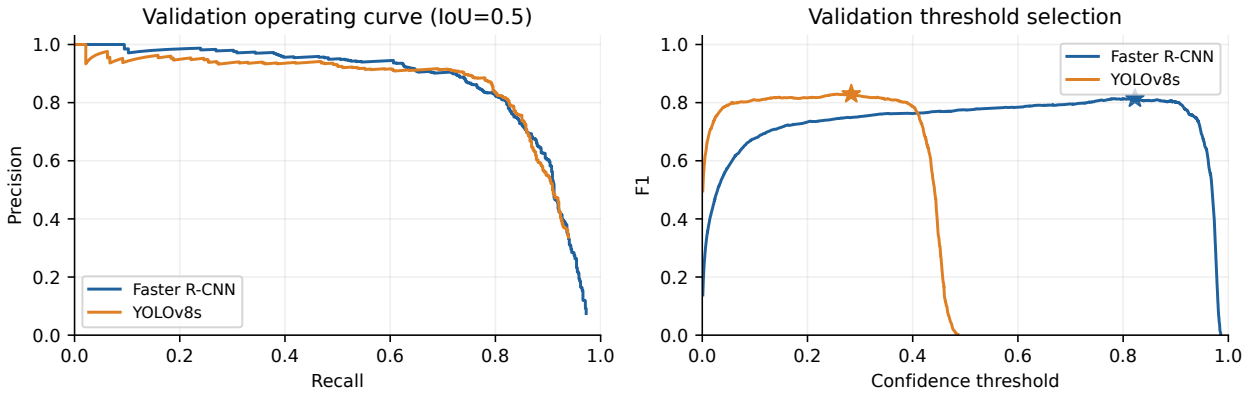
$$\text{FPS} = \frac{1000}{\bar{t}_{ms}}. \tag{11}$$

Số liệu là throughput suy ra từ latency batch 1, không phải đo video end-to-end, không gồm vẽ box/UI và không bảo đảm duy trì FPS khi dùng camera thực tế. VRAM inference là peak tensor allocation của Torch; không phải toàn bộ VRAM do driver/caching allocator chiếm và không phải yêu cầu GPU tối thiểu khi training.

7 Kết quả thực nghiệm

7.1 Validation và test

Bảng 4 cho thấy checkpoint được chọn theo AP, không theo F1. Faster R-CNN có AP validation tốt



Hình 2: Đường vận hành precision–recall và F1 theo threshold trên validation, ghép một-một ở IoU 0,5. Dấu sao chỉ threshold F1 được khóa cho test; đây không phải đường PR nội bộ của COCO ở mọi mức IoU.

Bảng 4: Validation tại best checkpoint và threshold đã chọn.

Chỉ số (%)	Faster R-CNN	YOLOv8s
AP@[.50:.95]	30,92	27,41
AP50	85,78	83,39
AP75	9,30	5,62
Precision	83,25	88,56
Recall	79,69	78,02
F1	81,44	82,95

Bảng 5: Test trên 3.408 ảnh, 1.068 ground-truth box. Bảng sinh từ metrics JSON.

Chỉ số test	Faster R-CNN	YOLOv8s
AP@[.50:.95] (%)	28,78	21,93
AP50 (%)	83,43	71,16
AP75 (%)	8,51	5,16
Precision (%)	82,00	80,15
Recall (%)	78,46	68,82
F1 (%)	80,19	74,06
Mean IoU TP (%)	68,89	67,45
AP small (%)	29,98	27,15
AP medium (%)	29,38	20,34
TP	838	735
FP	184	182
FN	230	333

hơn nhưng F1 validation thấp hơn YOLOv8s; đây không phải mâu thuẫn vì metric và tiêu chí lựa chọn khác nhau.

Trên test (Bảng 5), Faster R-CNN tăng 6,85 điểm phần trăm AP@[.50:.95], 12,28 điểm AP50, 9,64 điểm recall và 6,14 điểm F1 so với YOLOv8s. Precision tăng 1,84 điểm phần trăm. Đây là chênh lệch điểm phần trăm, không nhầm với phần trăm cải thiện tương đối. Ví dụ AP tăng tương đối khoảng 31,24%, nhưng 6,85 điểm phần trăm diễn đạt trực tiếp hơn khi so bảng.

Faster R-CNN có 838 TP, 184 FP, 230 FN; YOLOv8s có 735 TP, 182 FP, 333 FN. Thêm 103 TP với chỉ thêm 2 FP là lợi thế quan sát được tại hai threshold validation đã khóa. Tổng TP+FN đều bằng 1.068, phù hợp số ground truth. Test recall vẫn chỉ 78,46% ở hệ thống tốt hơn: còn khoảng 21,54% object bị bỏ sót.

7.2 Localization, kích thước và false alarm

AP75 chỉ đạt 8,51% và 5,16%, thấp hơn nhiều AP50. Mean IoU TP ở IoU 0,5 là 68,89% và 67,45%. Các số này cho thấy hạn chế khi yêu cầu box khớp chặt, dù model thường nhận diện đúng vùng lân cận Varroa ở chuẩn IoU 0,5. Không quy toàn bộ AP75 thấp cho nhãn thiếu chính xác; cần kiểm toán thêm cả annotation, độ phân giải nguồn, regressor và hậu xử lý để phân biệt nguyên nhân.

Test có tỷ lệ small box 768/1.068, khoảng 71,91%. AP small của Faster R-CNN là 29,98%, YOLOv8s 27,15%; AP medium lần lượt 29,38% và 20,34%. Như vậy, lợi thế AP medium lớn hơn lợi thế AP small trong run này; không dùng kết quả để khẳng định FPN chỉ cải thiện object nhỏ hay đã chứng minh nhân quả từ kiến trúc.

Trong 2.466 ảnh âm test, Faster R-CNN báo ít nhất một object ở 92 ảnh (3,73%); YOLOv8s ở 38 ảnh (1,54%). Mặc dù tổng FP gần nhau, phân bố lỗi khác nhau: Faster R-CNN làm phát sinh cảnh báo trên nhiều ảnh âm hơn. FP/image lần lượt 0,05399 và 0,05340. Nếu ứng dụng tốn công kiểm tra mỗi cảnh báo ảnh, false-alarm-image rate là chỉ số cần cân nhắc bên cạnh F1.

7.3 Tốc độ và tài nguyên

YOLOv8s nhanh hơn 3,40 lần, có ít hơn khoảng 3,71 lần parameter. Faster R-CNN có peak VRAM training ghi trong history khoảng 9.387 MiB/GPU (9,17 GiB), khác rõ với peak inference ở Bảng 6 vì training cần activation/gradient và batch lớn hơn. Artifact YOLO không cung cấp cùng trường peak training tổng hợp theo protocol này; không điền số ước đoán vào bảng để làm so sánh giả tương đương.

Thời gian training Faster R-CNN dài hơn dù số

Bảng 6: Hiệu suất inference với cùng protocol một T4, FP32, batch 1.

Chỉ số	Faster R-CNN	YOLOv8s
Parameter (triệu)	41,30	11,14
Mean latency (ms)	67,36	19,80
Median (ms)	67,33	19,59
P95 (ms)	71,20	21,62
FPS	14,84	50,51
Peak allocated (MiB)	365,41	89,58

epoch ít hơn, nhưng không thể giải thích chỉ bằng kiến trúc: Faster R-CNN dùng FP32, YOLOv8s AMP và EMA; best/dừng ở epoch khác nhau. Không dùng thời gian hai run để suy ra một hệ số chậm cố hữu cho mọi thiết lập.

8 Inference trên ảnh mới

8.1 Giao diện và đầu ra

Module `inference/predict.py` hỗ trợ một ảnh hoặc thư mục đệ quy, lựa chọn `fasterrcnn`, `yolov8s` hoặc `both`; device tự chọn CUDA khi có, nếu không dùng CPU. Notebook `inference/inference.ipynb` cho phép cấu hình source, output, model, device, limit và threshold rồi xem ảnh.

Inference tải `best.pt`, kiến trúc đúng head và FrozenBatchNorm cho Faster R-CNN; YOLO dùng model/EMA được lưu trong checkpoint. Không tải lại pretrained weights để thay thế weights fine-tune, không train lại. Ngưỡng được đọc từ `confidence_selection.json`; người dùng có thể override rõ ràng, khi đó phải coi đây là điểm vận hành mới, không gán các metric đã báo cáo cho threshold khác.

Mỗi model xuất ảnh gốc vẽ box/score, `predictions.json` theo COCO xywh và `run.json` chứa checkpoint, device, precision, threshold, manifest ảnh. ID trong inference bắt đầu từ 1 và chỉ có ý nghĩa trong lần chạy, không phải ID test gốc. Ảnh không có prediction vẫn giữ trong manifest. Khi chạy cả hai model, thư mục `comparison` chứa hai ảnh cạnh nhau để xem trực quan.

8.2 Ví dụ sử dụng

Các lệnh bên dưới chạy từ root repo Lab 3:

```
python inference/predict.py \
  --source /path/to/new_images \
  --model both --device auto

python inference/predict.py \
  --source /path/to/image.png \
  --model fasterrcnn --device cpu
```

Pipeline không gọi trực tiếp tiền xử lý default khác của `YOLO.predict()`; thay vào đó dùng cùng letterbox và predictor với final evaluation để tránh thay đổi hình học/normalization trong bước triển khai. NMS vẫn là 0,5, max detection 100, output box được clip trên kích thước ảnh gốc.

8.3 Kiểm chứng inference

Đã chạy cả hai checkpoint thật trên 4 ảnh test bằng CPU Torch 2.9.1 và Torchvision 0.24.1. Số detection khớp artifact Kaggle; chênh lệch box lớn nhất của Faster R-CNN khoảng $2,29 \times 10^{-5}$ pixel, của YOLOv8s khoảng $1,53 \times 10^{-5}$ pixel; chênh score YOLO khoảng $2,09 \times 10^{-7}$, Faster R-CNN bằng 0 trên các detection so khớp. Đây là smoke check 4 ảnh, không thay thế đánh giá toàn bộ test hoặc bảo đảm mọi ảnh/GPU đều có kết quả giống bit. Hình inference demo không chứa ground truth.

9 Phân tích định tính và thảo luận

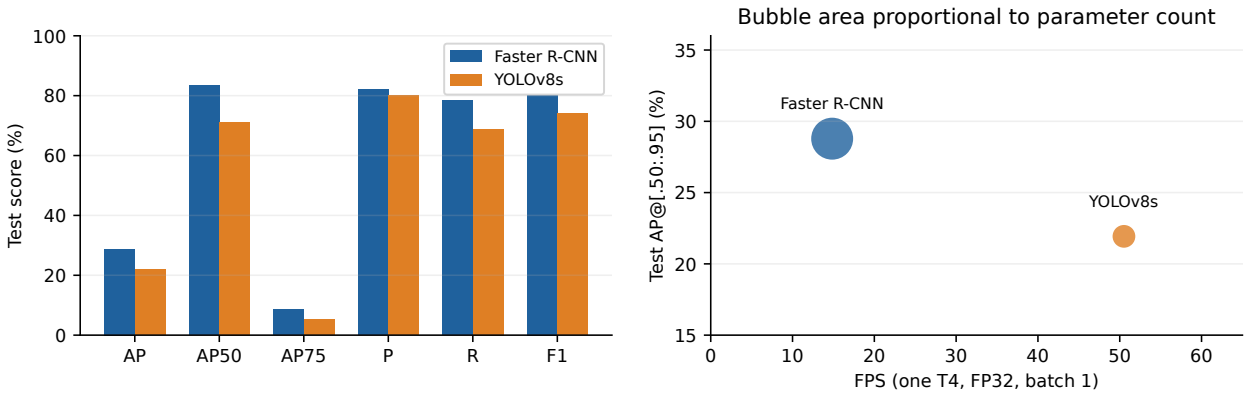
9.1 Trường hợp bổ sung recall và lỗi cảnh báo

Hình 4 minh họa một ảnh mà Faster R-CNN ghép đúng nhiều object hơn YOLOv8s; Hình 5 cho thấy cảnh báo trên ảnh được gán negative; Hình 6 là trường hợp cả hai không ghép đúng Varroa đã được gán nhãn. Ảnh được chọn bằng quy tắc đầu tiên thỏa từng điều kiện trong thứ tự test gốc, không chọn theo mức đẹp của hình. Manifest chọn mẫu được lưu để tái hiện. Mẫu ảnh minh họa không dùng để suy ra tần suất lỗi; tần suất lấy từ toàn bộ test.

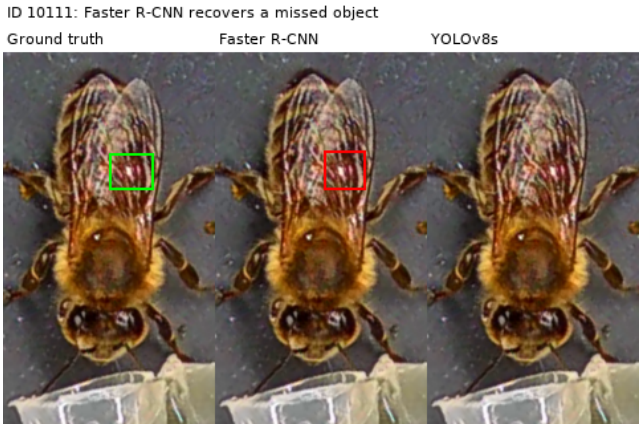
9.2 Overfitting và khoảng cách validation–test

Với Faster R-CNN, train loss giảm từ 0,0932 xuống 0,0373 nhưng validation AP sau epoch 7 dao động và không vượt 30,92%; cuối run còn 24,33%. Đây là dấu hiệu phù hợp với overfitting hoặc mất hiệu quả tổng quát hóa trong recipe hiện tại; chưa có ablation để tách tác động learning rate, augmentation và đặc điểm split. Early stopping cùng best checkpoint tránh dùng weights cuối kém hơn để báo cáo.

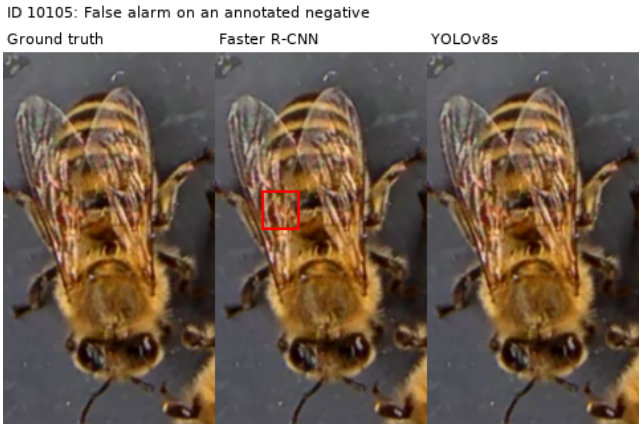
Với YOLOv8s, validation AP tăng dần rồi đạt đỉnh epoch 29, sau đó suy giảm nhẹ trong khi train loss tiếp tục giảm. Gap AP từ validation sang test của Faster R-CNN là 2,14 điểm, của YOLOv8s 5,47 điểm. YOLOv8s còn đổi thứ hạng F1: cao hơn trên validation nhưng thấp hơn trên test. Điều này cho thấy threshold chọn ở validation không bảo đảm tối ưu test, và không được chỉnh threshold bằng test để cải thiện lại số liệu cuối.



Hình 3: So sánh test và accuracy–speed. Diện tích điểm tỷ lệ parameter count. P/R/F1 dùng threshold riêng đã chọn trên validation; AP dùng protocol COCO chung.



Hình 4: Mẫu Faster R-CNN thu hồi đối tượng mà YOLO bỏ sót. Trái: ground truth xanh; giữa: Faster R-CNN; phải: YOLOv8s, prediction đỏ ở threshold validation riêng.



Hình 5: Một ảnh negative theo annotation có detection của Faster R-CNN và không có detection YOLO. "False alarm" được xác định đối với annotation, không khẳng định ảnh tuyệt đối không thể chứa đối tượng chưa được gán nhãn.

9.3 Lựa chọn hệ thống theo yêu cầu vận hành

Nếu ưu tiên thu hồi nhiều Varroa hơn trong hai run đã đo, Faster R-CNN là lựa chọn có cơ sở vì recall, AP và F1 cao hơn. Nếu ưu tiên latency hoặc tài nguyên inference, YOLOv8s có lợi thế. Nếu cảnh báo trên ảnh negative gây tốn công kiểm tra, YOLO có tỷ lệ ảnh false alarm thấp hơn tại điểm vận hành đã khóa. Do đó không có một lựa chọn "tốt nhất" chỉ dựa trên một cột mAP mà bỏ qua chi phí bỏ sót và chi phí cảnh báo.

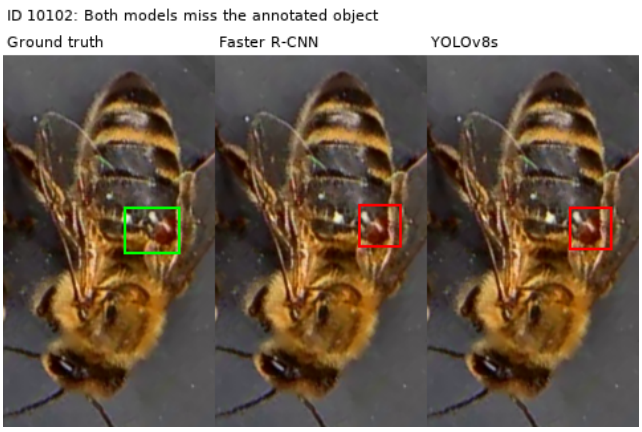
50,51 FPS của YOLO trong benchmark không tự chứng minh đáp ứng một hệ thống video thực tế. Cần đo thêm camera, decoding, hàng đợi, vẽ box, độ phân giải khác, nguồn điện và thermal throttling nếu chuyển sang thiết bị triển khai.

9.4 Giới hạn khoa học

Thứ nhất, một seed không đủ ước lượng biến thiên hoặc kiểm định ý nghĩa thống kê. Thứ hai, pretraining, learning rate, EMA, precision và pipeline riêng của backbone khác nhau; không coi đây là ablation kiến trúc được kiểm soát tuyệt đối. Thứ ba, dữ liệu crop trong môi trường kiểm soát chưa chứng minh hiệu quả ngoài hiện trường, ở camera hay colony khác. Thứ tư, split nguồn có thể còn tương quan phiên quay; hash/dHash chỉ là bằng chứng chống một số dạng trùng lặp. Thứ năm, evaluation chỉ dựa vào annotation hiện có; nhãn thiếu hoặc box chưa nhất quán có thể ảnh hưởng số đo. Thứ sáu, benchmark 100 ảnh không cung cấp phân phối latency cho toàn bộ dữ liệu hoặc nhiều thiết bị. Không cung cấp confidence interval vì chưa thực hiện ước lượng phù hợp.

9.5 Hướng phát triển

Các thực nghiệm tiếp theo nên tách thành giả thuyết kiểm chứng được: chạy nhiều seed; so sánh ở ngân sách/precision gần nhau; giảm learning rate hoặc thay schedule để khảo sát overfitting; ablation EMA và augmentation; kiểm toán các case IoU thấp; đánh giá trên tập ngoài phiên quay. Khảo sát threshold theo chi phí FN/FP hoặc recall mục tiêu phải thực hiện trên validation, sau đó khóa trước



Hình 6: Một ảnh có Varroa mà cả hai không ghép được TP tại IoU 0,5 và threshold đã chọn. Hộp dự đoán xuất hiện gần object cũng có thể bị tính FP nếu IoU không đủ.

test mới. Tăng input size không được mặc định là tạo lại chi tiết đã mất ở ảnh nguồn; cần đo thực nghiệm và so với chi phí latency/VRAM. Chưa có kết quả của các đề xuất này.

10 Kiểm thử và khả năng tái tạo

10.1 Kiểm thử đã thực hiện

Bộ kiểm thử hiện tại có 17 test pass trên CPU: round-trip box/letterbox; duplicate detection; empty predictions; oracle COCO; negative samples và batch YOLO; threshold khi chỉ có FP; metric CSV; DataLoader spawn/smoke subset; final evaluation; Faster R-CNN forward/backward trên batch trộn dương/âm và batch toàn âm; lỗi loss ở một rank Gloo làm cả hai rank dừng; cleanup process group; notebook nhúng source; và inference giữ tọa độ, ảnh rỗng, đường dẫn/filename. Không gọi kiểm thử Gloo CPU là xác nhận hiệu năng NCCL/T4; bằng chứng GPU là các run Kaggle đã hoàn thành.

Đã đọc lại best/last checkpoint của cả hai model, kiểm tra CRC ZIP/checkpoint; weights Faster R-CNN đều hữu hạn. Đã kiểm tra hash annotation giữa hai run giống nhau, đếm lại TP/FP/FN và tính lại AP bằng COCOeval từ exported predictions; kết quả khớp. Các bước kiểm tra này xác nhận tính nhất quán artifact, không phải một lượt huấn luyện độc lập hay bảo đảm tất cả lỗi triển khai đều đã bị loại.

10.2 Tổ chức artifact và tái lập báo cáo

Repo gồm source, notebook, tests, metadata dữ liệu, predictions, metrics, lịch sử và report. Dataset ảnh, checkpoint và ZIP nhiều GiB không đưa vào Git; giữ local và tải lại qua Kaggle nếu cần. LaTeX, hình và bảng được lưu trực tiếp để clean clone vẫn biên dịch PDF mà không tải checkpoint/dataset. `report/build_figures.py` sinh biểu đồ/bảng từ JSON/CSV; hình định tính cần ảnh dataset. `generated/artifact_sha256.json` lưu hash nguồn số liệu của báo cáo.

Hai run được lưu từ các kernel:

```
kaggle kernels output \
 mthutranhehe/lab3-yolov8 \
 -p output/yolo
kaggle kernels output \
 marcusnguyen007/lab3-faster-r-cnn \
 -p output/faster_r-cnn
```

Các đường dẫn download là phương thức lấy artifact đã báo cáo, không phải bằng chứng rằng run Kaggle tương lai sẽ không bị thay phiên bản bởi chủ kernel. Hash và metadata local giúp xác định bộ artifact thực tế trong bài này.

11 Kết luận

Bài lab hoàn thành pipeline từ kiểm toán Varroa-Dataset, huấn luyện hai detector đến evaluation và inference ảnh mới. Trong thiết lập một seed đã đo, Faster R-CNN ResNet50-FPN đạt AP@[.50:.95] 28,78%, F1 80,19%, recall 78,46% trên test, cao hơn YOLOv8s; YOLOv8s đạt 50,51 FPS so với 14,84 FPS của Faster R-CNN. Lợi thế recall của Faster R-CNN đi kèm chi phí tài nguyên/tốc độ và tỷ lệ ảnh negative bị cảnh báo cao hơn.

Khó khăn chính còn lại là localization ở IoU cao, bỏ sót đối tượng, overfitting và khả năng tổng quát hóa ngoài điều kiện dữ liệu. Kết quả cung cấp baseline có thể truy vết, cùng mô-đun inference thống nhất, và không vượt quá bằng chứng của hai run fine-tune cụ thể trong kết luận về các họ kiến trúc.

A Danh mục file để kiểm tra kết quả

Trong mỗi thư mục run: `config.json` là cấu hình; `environment.json` là phiên bản/GPU/hash; `metrics.json` chứa validation, test và benchmark; `comparison.csv` chứa bảy metric chính; `confidence_selection.json` lưu quy tắc/ngưỡng validation; `test_predictions.json` và `validation_predictions.json` chứa detection gốc ở score floor. `training_summary.json` lưu thời gian/epoch. History của Faster R-CNN là `history.csv`, của YOLO là `results.csv`.

Với Faster R-CNN, best/last nằm trực tiếp dưới run; với YOLO nằm trong `weights/`. Bản lỗi AMP cũ ở `buglog/lab3-faster-r-cnn.log`; log của run thành công nằm trong `output/faster_r-cnn/`. Không nhầm hai log này khi đánh giá trạng thái pipeline.

B Lệnh chạy và biên dịch

Từ root repo Lab 3, cài dependency inference và pytest trong môi trường có cặp Torch/Torchvision tương thích, rồi chạy:

```
python -m pytest tests -q
python inference/predict.py \
 --source /path/to/images --model both
cd report
make
```

PDF dùng XeLaTeX để hỗ trợ Unicode tiếng Việt với font Noto. Nếu không dùng Make, thứ tự là XeLaTeX, BibTeX rồi XeLaTeX hai lượt. Không cần Internet khi biên dịch nếu TeX, font và các package đã cài; hình/bảng đã có trong repo.

Tài liệu tham khảo

[1] Stefan Schurischuster and Martin Kampel. VarroaDataset, version 1.2.0. https://zenodo.org/records/4085044, 2020.

[2] Stefan Schurischuster. VarroaDataset: dataset documentation. https://github.com/schurist/VarroaDataset. Accessed 2026-10-25.

[3] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, 2015. https://arxiv.org/abs/1506.01497.

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2016. https://arxiv.org/abs/1512.03385.

[5] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2017. https://arxiv.org/abs/1612.03144.

[6] Ultralytics. YOLOv8 model configuration, release v8.3.228. https://github.com/ultralytics/ultralytics/blob/v8.3.228/ultralytics/cfg/models/v8/yolov8.yaml.

[7] Ultralytics. Official YOLOv8 documentation. https://docs.ultralytics.com/models/yolov8/. Accessed 2026-10-25; experiment uses Ultralytics 8.3.228.

[8] Ultralytics. Detection loss implementation, release v8.3.228. https://github.com/ultralytics/ultralytics/blob/v8.3.228/ultralytics/utils/loss.py.

[9] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision*, 2014. https://arxiv.org/abs/1405.0312.

[10] COCO Consortium. Official COCO evaluation implementation. https://github.com/cocodataset/cocoapi/blob/master/PythonAPI/pycocotools/cocoeval.py. Local evaluation: pycocotools 2.0.11.